P.Vishwa Nandan(192473015)

TASK 9 – Introduce Abstraction

Marks: 4

This task is about **Abstraction in Java** using abstract classes and interfaces.

Abstraction means hiding unnecessary implementation details and showing only the essential features.

In our Campus Smart-Parking System:

- An abstract class Vehicle represents common vehicle features.
- An interface Chargeable represents the charging ability of electric vehicles.

The assignment requires abstract classes, interfaces, multiple behaviours, default and static interface methods, and alternative concept analysis. CO1 JAVA

1. Abstract Class – Vehicle

An abstract class is a class that cannot be directly instantiated.

It can contain:

- Variables.
- Constructors.
- Normal methods.
- Abstract methods.

In our parking system, Vehicle is suitable as an abstract class because all vehicles share common properties, such as registration number and parking fee calculation.

Example:

```
abstract class Vehicle {

   String regNo;

   Vehicle(String regNo) {
      this.regNo = regNo;
   }

   abstract double ratePerHour();
```

```
    void display() {
        System.out.println("Vehicle Number: " + regNo);
    }
}
```

Here:

- Vehicle is an abstract class.
- regNo stores the vehicle registration number.
- ratePerHour() is an abstract method.
- display() is a normal method.

The abstract method must be implemented by its concrete subclasses.

2. Interface – Chargeable

An interface defines a set of behaviours that a class must implement.

In our parking system, electric vehicles require charging functionality.

Therefore, we create an interface called Chargeable.

Example:

```
interface Chargeable {

    double chargingFee(int kWh);
}
```

Here, chargingFee() calculates the electric vehicle charging cost.

The ElectricCar class implements this interface.

Example:

```
class ElectricCar extends Vehicle implements Chargeable {

    ElectricCar(String regNo) {
        super(regNo);
    }
```

```
    @Override
    double ratePerHour() {
        return 30;
    }

    @Override
    public double chargingFee(int kWh) {
        return kWh * 12;
    }
}
```

Here:

- ElectricCar inherits common properties from Vehicle.
- It implements the Chargeable interface.
- The charging fee is Rs. 12 per kWh.

3. Why Abstract Class and Interface Are Used

Both abstract classes and interfaces provide abstraction, but they serve different purposes.

Abstract Class – Vehicle

1. Stores common vehicle information.
2. Provides constructors for initializing vehicle objects.
3. Contains common methods.
4. Allows subclasses to implement abstract methods.

Interface – Chargeable

1. Defines charging behaviour.
2. Allows different classes to implement charging functionality.
3. Supports multiple interface implementation.
4. Separates charging functionality from general vehicle properties.

Therefore, an abstract class is suitable for shared properties, while an interface is suitable for additional capabilities.

4. Multiple Behaviours Through Interfaces

Java allows a class to implement multiple interfaces.

For example, an electric vehicle can support charging and tracking functionalities.

Example:

```
interface Chargeable {

    double chargingFee(int kWh);
}

interface Trackable {

    void updateLocation(String location);
}

class ElectricCar implements Chargeable, Trackable {

    public double chargingFee(int kWh) {
        return kWh * 12;
    }

    public void updateLocation(String location) {
        System.out.println("Location: " + location);
    }
}
```

Here, ElectricCar implements two interfaces:

- Chargeable provides charging functionality.
- Trackable provides vehicle location tracking.

This allows one class to obtain multiple behaviours without multiple class inheritance.

5. Default and Static Methods in Interfaces

Java interfaces can contain default and static methods.

A. Default Method

A default method contains an implementation inside an interface.

Example:

```
interface Chargeable {

    double chargingFee(int kWh);

    default void chargingInfo() {
        System.out.println("Electric vehicle charging is available");
    }
}
```

The implementing class can use this method without overriding it.

B. Static Method

A static method belongs to the interface itself and can be called using the interface name.

Example:

```
interface Chargeable {

    static boolean validateUnits(int kWh) {
        return kWh >= 0;
    }
}
```

Calling the method:

```
System.out.println(Chargeable.validateUnits(10));
```

Output:

```
true
```

Uses:

- Default methods provide common behaviour to implementing classes.
- Static methods provide utility operations related to the interface.

Explanation

1. Vehicle is an abstract class containing common vehicle information.
2. TwoWheeler and Car inherit from Vehicle.
3. ElectricCar inherits from Car.
4. ElectricCar implements Chargeable and Trackable interfaces.
5. The charging fee is calculated at Rs. 12 per kWh.
6. The default method displays charging information.
7. The static method validates the charging units.

This program demonstrates abstraction and multiple interface implementation.

---

7. Alternative Concept and Complexity Analysis

The original design uses:

- Vehicle as an abstract class.
- Chargeable as an interface.

The alternative design changes:

- Vehicle into an interface.
- Chargeable into an abstract class.

A. Vehicle as an Interface

An interface can define common vehicle behaviours.

Example:

```
interface Vehicle {

   String getRegNo();

   double ratePerHour();

   default double calculateFee(int minutes) {

      if (minutes <= 0) {
         return 0;
```

```
        }

        int hours = (minutes + 59) / 60;

        return hours * ratePerHour();
    }
}
```

Here, Vehicle provides common behaviour through a default method.

However, it cannot store an instance variable for the registration number or have a constructor.

Each implementing class must manage its own registration number.

B. Chargeable as an Abstract Class

An abstract class can contain charging-related fields, constructors and methods.

Example:

```
abstract class Chargeable {

    double ratePerUnit;

    Chargeable(double rate) {
        ratePerUnit = rate;
    }

    double chargingFee(int kWh) {
        return kWh * ratePerUnit;
    }
}
```

Here, the abstract class stores the charging rate and provides a common charging fee calculation.

C. Important Limitation

In Java, a class can extend only one class.

Therefore, if ElectricCar needs to extend both Car and Chargeable, it is not possible.

One possible redesign is:

- Car implements Vehicle.
- ElectricCar extends Chargeable and implements Vehicle.

However, ElectricCar can no longer inherit directly from Car. Any common car-specific behaviour must be duplicated or moved into composition.

D. Comparison

Original Approach:

- Vehicle is an abstract class.
- Chargeable is an interface.
- Vehicle can store common instance variables and constructors.
- ElectricCar can extend Car and implement Chargeable.
- Multiple interfaces can be implemented.

Alternative Approach:

- Vehicle is an interface.
- Chargeable is an abstract class.
- Vehicle cannot contain constructors or instance variables.
- Common vehicle data must be maintained in implementing classes.
- A class cannot extend both Car and Chargeable.

E. Complexity Analysis

Original Approach:

- Method execution time: O(1).
- Fee calculation time: O(1).
- Space per vehicle object: O(1).

Alternative Approach:

- Method execution time: O(1).
- Fee calculation time: O(1).
- Space per vehicle object: O(1).

Both approaches have similar asymptotic time and space complexity.

However, the alternative approach may require additional code to manage common vehicle information because interfaces cannot maintain instance state.

The original approach is easier to maintain for this parking system because it supports common vehicle data through an abstract class and additional capabilities through interfaces.

8. Conclusion

Abstraction separates common vehicle properties from specialised behaviours. The Vehicle abstract class stores common vehicle information, while the Chargeable interface provides charging functionality. Multiple interfaces allow a class to implement different behaviours. The original design is preferred because it provides better code reuse, flexibility and maintainability.